3D WEBSOFT INC. FOUNDATION

SCHOOL FUTURE TECH PROGRAM

Faculty Name :

Course Duration : 5 Months (150 Days)

Classes : Offline & Online

Daily Teaching : 3 Hours

Total Hours : 300 Hours

LEVEL 3 –Future Engineers(9th–10th Standard)

MODULE 1 – Python Foundations (Classes 1–10)

Class 8 – INPUT & OUTPUT

Complete Theory & Practical Notes

Learning Objectives

By the end of this session, students will be able to:

1. Understand the meaning of input and output in programming.
2. Explain how a Python program receives information from a user.
3. Understand the purpose of the print() function.
4. Use print() to display text and values.
5. Understand the purpose of the input() function.
6. Take user input using input().
7. Store user input inside variables.
8. Understand that input() normally returns text (str).
9. Use int() and float() to convert suitable user input into numbers.
10. Combine text, variables, and user input in output statements.
11. Create interactive Python programs.
12. Develop problem-solving and logical-thinking skills through practical activities.

Session

Topic	*Type*
Warm-up: How Do Programs Communicate?	Discussion
Understanding Input & Output	Theory
Output Using `print()`	Theory + Demonstration
Printing Variables and Multiple Values	Demonstration + Practice
Input Using `input()`	Theory + Demonstration
Personal Information Program	Hands-on Practical 1
Break / Input-Output Challenge	Activity
Understanding `input()` and Strings	Theory + Demonstration
Converting Input Using `int()` and `float()`	Theory + Practical
Student Details Program	Hands-on Practical 2
Quiz + Exit Check	Assessment

1. Warm-up – How Do Programs Communicate?

"How does a computer program communicate with a person?"

Ask students to think about everyday applications.

Examples:

- A calculator asks for numbers.
- A login page asks for a username and password.
- A game asks the player for a name.
- A shopping application asks for product information.
- A form asks for your age and address.

Explain that programs generally need two important things:

Input

Information **given to the program**.

Output

Information **produced or displayed by the program**.

Introduce the simple programming flow:

```
 USER
   |
   ↓
INPUT
   |
   ↓
PROGRAM
   |
   ↓
OUTPUT
   |
   ↓
 USER
```

Simple Example

Imagine a program asks:

```
What is your name?
```

The user enters:

```
Rahul
```

The program responds:

```
Hello Rahul!
```

Here:

```
Rahul       → Input
Hello Rahul → Output
```

2. Understanding Input & Output

THEORY

What is Input?

Input is information supplied to a computer program.

Examples:

```
Name
Age
Password
Marks
City
Temperature
Choice
```

In Python, one of the easiest ways to receive input from a user is:

```
input()
```

What is Output?

Output is information displayed or produced by a program.

In Python, the most common function used to display output is:

```
print()
```

Therefore, introduce the two important functions:

```
input()
print()
```

Remember

input() → Gets information from the user

print() → Displays information to the user

Input → Processing → Output

```
INPUT
  ↓
PROCESS
  ↓
OUTPUT
```

For example, a program that calculates the total of two numbers:

```
User enters two numbers
        ↓
Python adds them
        ↓
Program displays total
```

This concept will become increasingly important as students learn operators and calculations.

3. Output Using print()

THEORY + DEMONSTRATION

What is `print()`?

The `print()` function displays information on the screen.

Example 1 – Printing Text

```
print("Hello Python")
```

Output:

```
Hello Python
```

Example 2 – Printing a Number

```
print(100)
```

Output:

```
100
```

Example 3 – Printing a Decimal

```
print(25.5)
```

Output:

```
25.5
```

Example 4 – Printing Boolean Values

```
print(True)
```

Output:

```
True
```

Printing Different Types

```
print("Python")
print(25)
print(25.5)
print(True)
```

Output:

```
Python
25
25.5
True
```

Connect this to the previous **Data Types** class:

```
"Python" → str
25       → int
25.5     → float
True     → bool
```

4. Printing Variables

DEMONSTRATION + PRACTICE

Students already know variables from previous classes.

Explain that print() can display the value stored inside a variable.

Example

```
name = "Rahul"

print(name)
```

Output:

```
Rahul
```

Multiple Variables

```
name = "Rahul"
age = 15
city = "Hyderabad"

print(name)
print(age)
print(city)
```

Output:

```
Rahul
15
Hyderabad
```

Printing Labels with Variables

Instead of:

```
print(name)
print(age)
```

we can make the output easier to understand:

```
print("Name:", name)
print("Age:", age)
```

Output:

```
Name: Rahul
Age: 15
```

More Examples

```
name = "Anita"
age = 14
school = "Future School"

print("Student Name:", name)
print("Age:", age)
print("School:", school)
```

Output:

```
Student Name: Anita
Age: 14
```

```
School: Future School
```

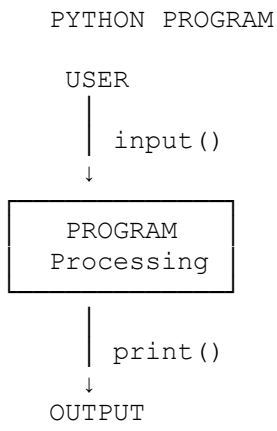
Mini Practice

Ask students to create:

```
name = "_________"
age = _________
city = "_________"

print("Name:", name)
print("Age:", age)
print("City:", city)
```

Suitable Visuals – Input & Output



5. Input Using input()

THEORY + DEMONSTRATION

Now introduce the input() function.

Explain:

`input()` allows a Python program to ask the user for information.

Basic Example

```
name = input("What is your name? ")

print("Hello", name)
```

Possible interaction:

```
What is your name? Rahul
Hello Rahul
```

Explain what happened:

```
input() → asks the question
Rahul   → user enters the information
name    → stores the information
print() → displays the result
```

Input Flow

```
input()
   ↓
User enters information
   ↓
Information is stored in variable
   ↓
print()
   ↓
Output displayed
```

Example 2 – Asking for City

```
city = input("Enter your city: ")

print("You live in", city)
```

Possible output:

```
Enter your city: Hyderabad
You live in Hyderabad
```

Example 3 – Asking for Favorite Subject

```
subject = input("What is your favorite subject? ")

print("Your favorite subject is", subject)
```

Possible output:

```
What is your favorite subject? Science
Your favorite subject is Science
```

6. Personal Information Program

HANDS-ON PRACTICAL 1

Students create a file:

`personal_info.py`

Starter Program

```
name = input("Enter your name: ")
city = input("Enter your city: ")
school = input("Enter your school: ")
subject = input("Enter your favorite subject: ")

print()
print("===== MY INFORMATION =====")
print("Name:", name)
print("City:", city)
print("School:", school)
print("Favorite Subject:", subject)
```

Example Run

```
Enter your name: Rahul
Enter your city: Hyderabad
Enter your school: Future School
Enter your favorite subject: Science

===== MY INFORMATION =====
Name: Rahul
City: Hyderabad
School: Future School
Favorite Subject: Science
```

Student Challenge

Ask students to add two more questions:

```
favorite_color = input("Enter your favorite color: ")
hobby = input("Enter your hobby: ")
```

Then display them:

```
print("Favorite Color:", favorite_color)
print("Hobby:", hobby)
```

7. Break / Input-Output Challenge

75–83 Minutes

7. Break / Input-Output Challenge

Give students a short challenge.

Display:

```
Program asks:
"What is your name?"

User enters:
Anita

Program displays:
"Welcome Anita!"
```

1. What is the input?
2. What is the output?
3. Which Python function gets the input?
4. Which Python function displays the output?

Expected Answers

```
Input  → Anita
Output → Welcome Anita!
Input function  → input()
Output function → print()
```

Pair Activity

Student A acts as the **user**.

Student B acts as the **programmer**.

Student B asks:

```
What is your name?
What is your favorite food?
What is your hobby?
```

Student A answers.

Then swap roles.

This helps students understand the difference between **user input** and **program output**.

8. Important Concept – input() Returns Text

THEORY + DEMONSTRATION

This is an important concept for beginners.

Explain:

By default, the value returned by input() is a **string (str)**.

Example

```
age = input("Enter your age: ")

print(age)
print(type(age))
```

If the user enters:

```
15
```

the output will be similar to:

```
15
<class 'str'>
```

Explain:

Even though the user typed 15, Python initially receives it as text.

Compare:

```
age = 15
```

with:

```
age = input("Enter your age: ")
```

The first contains an integer:

```
15 → int
```

The second normally contains a string:

```
"15" → str
```

Why Does This Matter?

Suppose we write:

```
age = input("Enter your age: ")

print(age + 1)
```

This does **not** work as intended because age is text.

We need to convert the input into a number.

Introduce:

```
int()
```

9. Converting Input Using int()

THEORY + PRACTICAL

String to Integer

Use:

```
int()
```

Example:

```
age = int(input("Enter your age: "))

print("Next year you will be", age + 1)
```

If the user enters:

```
15
```

Output:

```
Next year you will be 16
```

Explain the process:

```
User enters "15"
        ↓
input() receives text
        ↓
int() converts it
        ↓
15 becomes an integer
        ↓
Python can perform arithmetic
```

Example – Two Numbers

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

total = num1 + num2

print("Total:", total)
```

Example:

```
Enter first number: 20
Enter second number: 30

Total: 50
```

Converting Decimal Input

For decimal numbers, use:

```
float()
```

Example:

```
height = float(input("Enter your height: "))

print("Your height is", height)
```

If the user enters:

```
160.5
```

Python stores it as a float.

`int()` VS `float()`

Function	Purpose	Example
`int()`	Converts suitable input to integer	`"25"` → `25`
`float()`	Converts suitable input to float	`"25.5"` → `25.5`
`str()`	Converts a value to string	`25` → `"25"`

```
int()
float()
```

10. Student Details Program

HANDS-ON PRACTICAL 2

Students create:

`student_details.py`

Starter Code

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
height = float(input("Enter your height: "))
favorite_subject = input("Enter your favorite subject: ")

print()
print("===== STUDENT DETAILS =====")
print("Name:", name)
print("Age:", age)
print("Height:", height)
print("Favorite Subject:", favorite_subject)
```

Example Run

```
Enter your name: Anita
Enter your age: 14
Enter your height: 155.5
Enter your favorite subject: Mathematics

===== STUDENT DETAILS =====
Name: Anita
Age: 14
Height: 155.5
Favorite Subject: Mathematics
```

Type Checking Challenge

Add:

```
print()
print("===== DATA TYPES =====")
print(type(name))
print(type(age))
print(type(height))
print(type(favorite_subject))
```

Students should observe:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'str'>
```

Connect this activity directly to the previous class on **Data Types**.

11. Final Practical Challenge – Simple Calculator

If students finish early, give them this challenge.

Create:

`simple_calculator.py`

Program

```
num1 = int(input("Enter first number: "))
num2 = int(input("Enter second number: "))

print("First number:", num1)
print("Second number:", num2)
print("Sum:", num1 + num2)
```

Example

```
Enter first number: 25
Enter second number: 15

First number: 25
Second number: 15
Sum: 40
```

Extension

Ask students to add subtraction:

```
print("Difference:", num1 - num2)
```

Then multiplication:

```
print("Product:", num1 * num2)
```

This provides a bridge to the upcoming **Operators** lessons.

12. Quick Assessment & Exit Check

Question 1

Which function is used to display output?

Answer:

```
print()
```

Question 2

Which function is used to receive user input?

Answer:

```
input()
```

Question 3

What does this program do?

```
name = input("Enter your name: ")
print("Hello", name)
```

Answer: It asks the user for their name and then displays a greeting.

Question 4

What type of value does input() normally return?

Answer:

```
Str
```

Question 5

How can we convert suitable user input into an integer?

Answer:

```
int()
```

Question 6

How can we convert suitable user input into a decimal number?

Answer:

```
float()
```

Question 7

What will happen here?

```
age = int(input("Enter age: "))
print(age + 1)
```

If the user enters:

```
14
```

Answer:

```
15
```

13. Classroom Game – "Input or Output?"

Display different examples and ask students to identify whether each one is **INPUT** or **OUTPUT**.

Example	Answer
User enters their name	Input
`input("Enter age: ")`	Input
`print("Hello")`	Output
Program displays a total	Output
User enters a city	Input
`print(age)`	Output

Fast Round

Teacher says:

"User enters 25."

Students answer:

INPUT!

Teacher says:

"Program displays 50."

Students answer:

OUTPUT!

Teacher says:

"Which function?"

Students answer:

`input()` / **`print()`**

14. Common Beginner Mistakes

Mistake 1 – Forgetting quotation marks

Incorrect:

```
print(Hello)
```

Correct:

```
print("Hello")
```

Mistake 2 – Forgetting parentheses

Incorrect:

```
print "Hello"
```

Correct:

```
print("Hello")
```

Mistake 3 – Forgetting to store input

Students may write:

```
input("Enter your name: ")
```

This asks for input, but if they want to use the answer later, they should store it:

```
name = input("Enter your name: ")
```

Mistake 4 – Treating numeric input as an integer automatically

```
age = input("Enter age: ")
```

`age` is normally a string.

For a numeric age:

```
age = int(input("Enter age: "))
```

Mistake 5 – Entering letters where `int()` expects a number

For example:

```
age = int(input("Enter age: "))
```

If the user enters:

```
fourteen
```

Python cannot convert that text into an integer.

For this beginner lesson, simply explain that int() requires input that represents a valid integer.

15. Teacher Demonstration Checklist

Before class, make sure the teacher can demonstrate:

```
print("Hello Python")
name = "Rahul"
print("Hello", name)
name = input("Enter your name: ")
print("Hello", name)
age = int(input("Enter your age: "))
print("Next year:", age + 1)
height = float(input("Enter your height: "))
print("Height:", height)
```

16. Key Concepts for the Board

At the end of the lesson, write this summary on the board:

```
               INPUT & OUTPUT

INPUT
↓
input()

Gets information from the user


OUTPUT
↓
print()

Displays information on the screen


NUMERIC INPUT
↓
int()   → Whole number
float() → Decimal number


IMPORTANT:
input() normally returns a string.
```

Learning Outcomes

By the end of this class, students will be able to:

1. **Define** input and output in programming.
2. **Explain** how users interact with Python programs.
3. **Use** the print() function to display information.
4. **Display** text, numbers, Boolean values, and variables using print().
5. **Use** labels with variables to create clear output.
6. **Use** the input() function to receive information from users.
7. **Store** user input in variables.
8. **Explain** that input() normally returns a string (str).
9. **Use** int() to convert suitable user input into integers.
10. **Use** float() to convert suitable user input into decimal numbers.
11. **Differentiate** between text input and numeric input.
12. **Use** type() to check the type of values received from input.
13. **Create** an interactive Personal Information program.
14. **Create** an interactive Student Details program.
15. **Apply** input, variables, data types, and output together in a simple Python program.
16. **Create** a basic calculator program using user input and arithmetic operations.
17. **Identify and correct** common beginner mistakes involving input() and print().